The live demonstration: what you need to know

Presenter guide for the PQC hackathon jury session (Phase 1: 3-min pitch, 5–7 min deep dive, 10 min Q&A). All numbers come from our own runs on this board.

1. What the demo proves

The jury wants to see one thing: an ECC device has been migrated to post-quantum protection on **both planes**, and the protection actually works. The demo shows it live on the real board, step by step:

- **Boot plane.** On every reset the Edge Protect Bootloader (EPBL) hashes the application with SHA-512 and verifies its **ML-DSA-87** signature against a key compiled into EPBL. Only then does the application run.
- **Attacks are rejected.** An update signed with an attacker's key, an update with one changed byte, and a genuine but older version (rollback) are all refused, and the verified image keeps running.
- **Secure update.** A genuine newer version (v1.1.0) is verified, installed and started.
- **Runtime plane.** The PC (the SPDM requester) authenticates the running chip with SPDM 1.4: an ML-DSA-87 certificate chain plus **signed measurements** of the firmware. It then opens an encrypted session using **ML-KEM-1024** and AES-256-GCM, after the update. That covers the brief's "reconnect after update".

This covers every bullet of the brief's live-demonstration list: authorized boot, rejected tampering, a valid update, SPDM authentication, a protected session, encrypted application data, and reconnect after update.

2. Before you walk in (about 10 minutes before the slot)

- **Cables:** the board's USB J13 (J-Link + COM4 console) and the KitProg3 I2C bridge (wires SDA J16.6, SCL J16.5, VTARG J15.7, GND J15.8). The I2C bridge is only needed for SPDM.
- **Close every serial monitor** (ModusToolbox terminal, PuTTY, Tera Term). If COM4 is taken, the app now says "COM4 could not be read" instead of showing the board output.
- Double-click **tools\demo_app\START DEMO.cmd**. A console window starts the server and Chrome opens full screen. Keep the console window open: closing it stops the demo.
- Check the three status chips at the top right: **J-Link, COM4, I2C**. The "installed" field shows the version read from flash.
- Press **P** (pre-flight) about **8 minutes** before the jury. It programs EPBL + responder v1.0.0 (about 20 s; the chip's identity key is not touched), then runs one **live SPDM attestation** (about 7 min). When the screen says **READY**, press **Home**. Then wait. Press **Space** only when the deep dive starts.

The one rule: a board reset kills a running SPDM operation. While a live SPDM run is going, the app refuses board steps (it shows a message). Press **A** to abort the live run first if you really must. Never use "Erase device" or the provisioning **--freeze** option: this is a shared board.

3. Controls

Key	Action	Key	Action
Space	start / pause the automatic run	R	redo the current step (always safe)
→ / ←	next / previous step	1 – 8	jump to a step
Home or 0	back to the start screen	P	pre-flight (known state + live attestation)
A	abort a live SPDM run	L	LED test (LED4 blinks)
F	toggle full screen	Alt+F4	close Chrome (server keeps running)

If Chrome is closed by accident: **OPEN UI ONLY.cmd**. There is also a **simple** version (**START SIMPLE DEMO.cmd**, or http://127.0.0.1:8765/simple): one button per experiment, which starts immediately. It also has a "header edited" attack and 20-second SPDM replays. Use it in Q&A when a juror asks "can you show X again?".

4. The run, step by step

After Space the run advances by itself (about 3.5 minutes for steps 0–6). You can pause at any point to talk. For each step: what happens, what to say, and what the screen should show.

Step 0 · Overview (14 s)

No board action. The screen shows the chain of trust (ROM_BOOT → FLASH_BOOT → EPBL → responder → host) and the CNSA 2.0 mapping table. Note the two grey "classical" boxes: you will come back to them.

> *"An ECC-based SPDM device, migrated to the CNSA 2.0 profile. Everything you see next runs live on this board."*

Step 1 · Verified boot (expect: VERIFIED · RUNNING, ~4 s)

The app resets the board. The console shows the real EPBL log from COM4, ending with "validated successfully" and the responder banner. The INPUT panel shows what EPBL checks: the SHA-512 stored in the image versus recomputed, the signer key hash versus the key built into EPBL, and the 4,627-byte ML-DSA-87 signature. The check takes about **80 ms** (from the EPBL log timestamps).

> *"EPBL hashes the whole 165 KB image with SHA-512 and verifies the ML-DSA-87 signature before it jumps to the application."*

Step 2 · Attack: rogue key (expect: ATTACK BLOCKED, ~12 s)

A perfectly formatted update, signed with a real ML-DSA-87 key, but the attacker's key. The key-hash row turns red. EPBL rejects it in about 20 ms, LED4 blinks fast, and "installed" stays v1.0.0.

> *"Being well-formed is not enough. Only our provisioned key may sign firmware."*

Step 3 · Attack: one byte (expect: ATTACK BLOCKED, ~13 s)

A genuine signed update with one code byte changed after signing. The right panel shows the exact byte, old versus new value. The recomputed SHA-512 no longer matches the signed digest.

> *"One bit is enough. The fingerprint changes completely, so the signature no longer fits."*

Step 4 · Valid update v1.1 (expect: UPDATE INSTALLED, ~14 s)

A genuine newer image. EPBL verifies the candidate, copies it into the primary slot, verifies it again and starts it. "installed" changes to v1.1.0. The SPDM identity in protected storage is not touched.

> *"This is the normal field update: verified twice, and the device keeps its identity."*

Step 5 · Attack: rollback (expect: ATTACK BLOCKED, ~8 s)

The genuine but older v1.0.0. Signature and digest are fine; the only defence is the version check. EPBL logs "insufficient version".

> *"The classic downgrade attack: an old version with a known bug, correctly signed by us. Refused because it is older."*

Step 6 · SPDM attestation (replay, ~20 s)

This is a fast replay of the live run from your pre-flight: the real bytes from this board, 187 transfers. You see every SPDM message decoded, the ML-DSA-87 OID in the certificate bytes, and at the end three green **MATCH** rows. The device's signed measurements (EPBL digest, verified-image digest, version) equal what the PC recomputes from the release files.

> *"The host pins an ML-DSA-87 root, checks the 15 KB chain, and the chip attests exactly the image EPBL verified. This was recorded live 10 minutes ago. Over this debug bridge one run takes 7 minutes, so we replay it fast."*

Step 7 · Encrypted session (live, ~7 min, runs in the background)

Now a **live** SPDM session starts against the updated v1.1.0: ML-KEM-1024 key exchange, transcript signed with ML-DSA-87, then a time-stamped message sent AES-256-GCM-encrypted and echoed back. The screen shows the ciphertext on the wire. Move on; a pop-up announces the result during Q&A (press 7 to view it).

> *"This is the reconnect after the update. The device needs under a second; the 7 minutes are the 127-byte I2C test tool."*

Step 8 · Evidence & cost

Recorded results: the Task 4 lifecycle run (18/18) and the Task 3 SPDM tamper campaign (12/12 rejected), the key cost numbers, and the "Honest limits" box. End here and go to Q&A.

> *"PQC compute is not the bottleneck on this Cortex-M33. Bytes, RAM and transport are. And this is what is still classical."*

5. How to read the screen

- **Verdict card (top right):** green = verified/installed, amber "ATTACK BLOCKED" = rejected as intended, red = something unexpected. A red line "expected: ..." means the result was not what the step wanted.
- **Board console:** the unedited EPBL and responder output from COM4. This is your proof that it is not a simulation. Lines in green/red are the decisive ones.
- **LED3** (physical, on the board) is the responder's heartbeat: it blinks while idle. **LED4** is driven by the app: steady = verified image running, fast blink = attack rejected, off = reset, flicker = SPDM traffic. The LED icons on screen are read from the real GPIO register, not simulated. Pointing at the physical board is effective.
- **SPDM view:** transfer counter, bytes on the bus, every message with request (→) and response (←) bytes. Encrypted messages are marked; nobody on the bus can read them.

6. When something goes wrong

Symptom	What to do
"J-Link could not reach the board"	Check USB J13, then press R. If it persists: close the app, replug, START DEMO.cmd.
"COM4 could not be read"	A serial monitor is open: close it, press R.
NO BOOT / board silent	Press P, or use "Known state" in the simple UI (20 s). The identity is never touched.
"No I2C bridge found"	Replug the KitProg3 USB, wait 5 s, retry. Step 6 still works: it is a replay.
Board step refused: "SPDM running"	Intended. Wait for the live run, or press A to abort it.
Live session fails	Say so calmly: it fails closed, with no insecure fallback. Show step 6 and step 8 (recorded 3/3 exchanges, 12/12 rejected).
Everything breaks	Backup: tools\demo*.cmd buttons, and "4 RECORDED evidence" shows the recorded results instantly.

Tested today 11:06 on the board: prepare, boot, rogue, one byte, header, valid, rollback and the SPDM replay all gave the expected verdicts. A busy COM4 and a failed SPDM start now show clear errors instead of misleading results.

7. Questions the jury will likely ask

Q: Is the whole boot chain quantum-safe?

No, and we say so. ROM_BOOT is immutable silicon and only does ECDSA. On our board FLASH_BOOT does not even check EPBL (SIMPLE_APP policy): enabling that needs an Infineon-signed OEM certificate and steps we could not do on a shared board. Everything from EPBL onward is PQC. A fully PQC root needs new silicon; meanwhile the ECDSA ROM step protects a small, rarely changed EPBL.

Q: Why ML-DSA-87 and not ML-DSA-65?

CNSA 2.0 requires NIST category 5. ML-DSA-65 is category 3. We tested that the device rejects ML-DSA-65 at boot, in SPDM and in provisioning.

Q: Why not LMS/XMSS for firmware?

CNSA 2.0 allows them, but they are stateful: the signer must never reuse a one-time key, which is risky in a build pipeline. ML-DSA-87 is stateless and is the same algorithm as the SPDM identity, so one PQC family covers both planes.

Q: Why is PQC verification faster than ECDSA?

On this chip P-384 runs in software (only P-256 is accelerated). ML-DSA verification is fast integer arithmetic. Caution: the 170 ms ECDSA figure was measured on the smaller demo app (ML-DSA there: 61 ms); on the 165 KB responder ML-DSA takes about 80 ms.

Q: Why does SPDM take 7 minutes?

The host test tool starts a new process for every 127-byte I2C transfer (~1.4 s each), and a PQC session needs about 200 transfers. On the device the whole handshake is a few hundred milliseconds: ML-KEM encapsulation 11 ms versus 163 ms for ECDHE. A real transport (PCIe/MCTP) removes this.

Q: Where is the private key? What if EPBL is bypassed?

The ML-DSA-87 key is generated on the chip and stays in the TrustZone secure world. Malicious firmware that bypassed EPBL could not fake the signed measurements without also compromising the secure world; in production, BootROM authentication of EPBL closes that path.

Q: Can I flash an old image with a debugger?

Today yes: rollback protection is a software version check on the update path. Production needs the eFuse anti-rollback counter and a closed debug port. Both are irreversible, so we did not do them on a shared board.

Q: What does it cost?

Mostly RAM and bytes, not CPU or flash: certificate chain 1.2 KB → 15.2 KB, session traffic 2.6 KB → 28 KB (24 → 214 transfers), runtime buffers 20 KB → 58 KB, boot RAM 10.7 KB → 25.2 KB, image metadata 215 B → 4.8 KB. Responder flash stayed the same (130 → 128 KB).

Q: How do you migrate devices already in the field?

Five steps: field-update the firmware (still verified by the ECDSA EPBL); replace EPBL with the ML-DSA-87 EPBL; generate a new ML-DSA-87 identity on the device, vouched for by the old ECC identity; switch to the PQC-only profile; requesters drop the ECC root. We executed steps 1–4 on this board across the tasks.

Q: Why no hybrid (ECC + PQC)?

The brief excluded it. A PQC-only profile also avoids any downgrade path: the device offers no classical algorithm at all.

8. Numbers to have in your head

	ECC baseline	PQC (this demo)
Boot signature check	ECDSA P-384, 170 ms	ML-DSA-87 + SHA-512, 61 ms (demo app) / ~80 ms (responder)
Reset to application	266 ms	146 ms
Rejected attack at boot	–	about 20 ms, the old image keeps running
Device signature (SPDM)	84 ms	66–346 ms, median 112 ms (varies: rejection sampling)
Key exchange on device	ECDHE 163 ms	ML-KEM-1024 encapsulation 11 ms
Certificate chain	1,218 B	15,187 B (fills the 15 KB slot to 99 %)
Session on the wire	2,569 B, 24 transfers	28,199 B, 214 transfers
Tests on hardware	6/6 tamper cases rejected	Boot 11/11 rejected, SPDM 12/12 rejected, lifecycle 18/18

Your closing line: "Everything from the bootloader to the encrypted session is post-quantum, measured on a 128 KB-RAM microcontroller. What remains classical is the immutable ROM, and we have shown exactly what it takes to close that gap."